

Introduction to Retrieval Augmented Generation (RAG)

Tech-talk Presented By: Marc Van Goolen

Organization: Bencode

Contact: marc.vangoolen@telenet.be

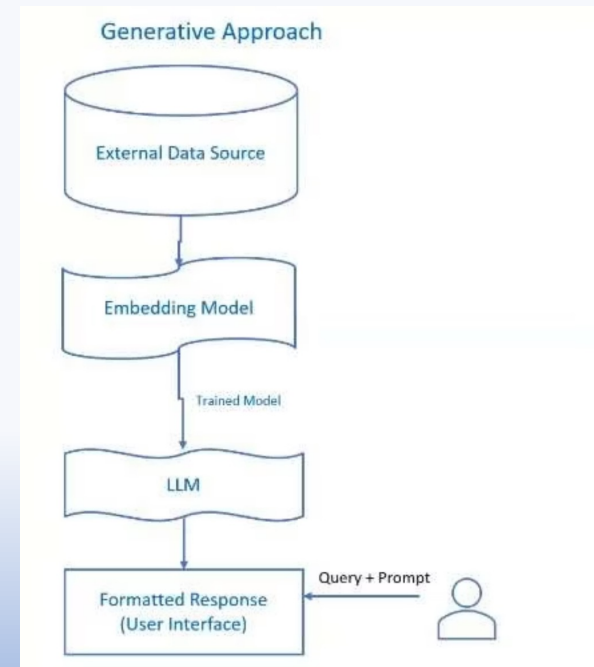
Agenda

1. Intro LLM + Limitations
2. What is RAG and Why is it important?
3. RAG Architecture & Flow
4. RAG Major Applications
5. RAG Chat Application
6. Wrap-up Benefits of RAG
7. Useful Links!!!

What is an LLM (in-short) ?

- AI program that can recognize and generate text, images, among other tasks (long list!).
- LLM are very large models that are pre-trained on vast amounts of data.
- Built on transformer architecture is a set of neural network that consist of an encoder and a decoder with self-attention capabilities.
- It can perform completely different tasks such as answering questions, summarizing documents, translating languages and completing sentences.

Open AI's **GPT-3** model has **175 billion** parameters. Also it can take inputs up to 100K tokens in each prompt. **GPT-4** Estimated Parameters : Estimated to have **~1.8 trillion parameters**
Architecture reportedly consists of **8 models**, each with **220 billion parameters**



LLM's - Limitations

- **Not Updated to the latest information** - only to date they are trained.
- **Hallucinations:** Output which is factually incorrect or nonsensical, but looks coherent and grammatically correct.
- **Domain-specific most accurate information:** output lacks accurate information when specificity is more important than generalized output
- **Source Citations:** In GenAI responses, we don't know what source it is referring to generate a particular response
- **Updates take Long training time:** requires huge resources and long training time which is a computationally intensive task.

What is RAG?

- Combines search and generation
- Queries external documents in real time
- Adds retrieved data to the prompt context
- Generates answers that are more accurate and grounded
- Solves the issue of stale model knowledge
- Huge step forward with regard to GenAI safety and usefulness.”



You

Who Won Google Cloud Technology Partner of Year (2023) Award for AI and Machine Learning



ChatGPT

I can't provide real-time information, including award winners beyond my last update in January 2022. To find out who won the Google Cloud Technology Partner of the Year Award for AI and Machine Learning in 2023, you should check the latest announcements from Google or reputable tech news sources. They often cover such awards and recognitions.

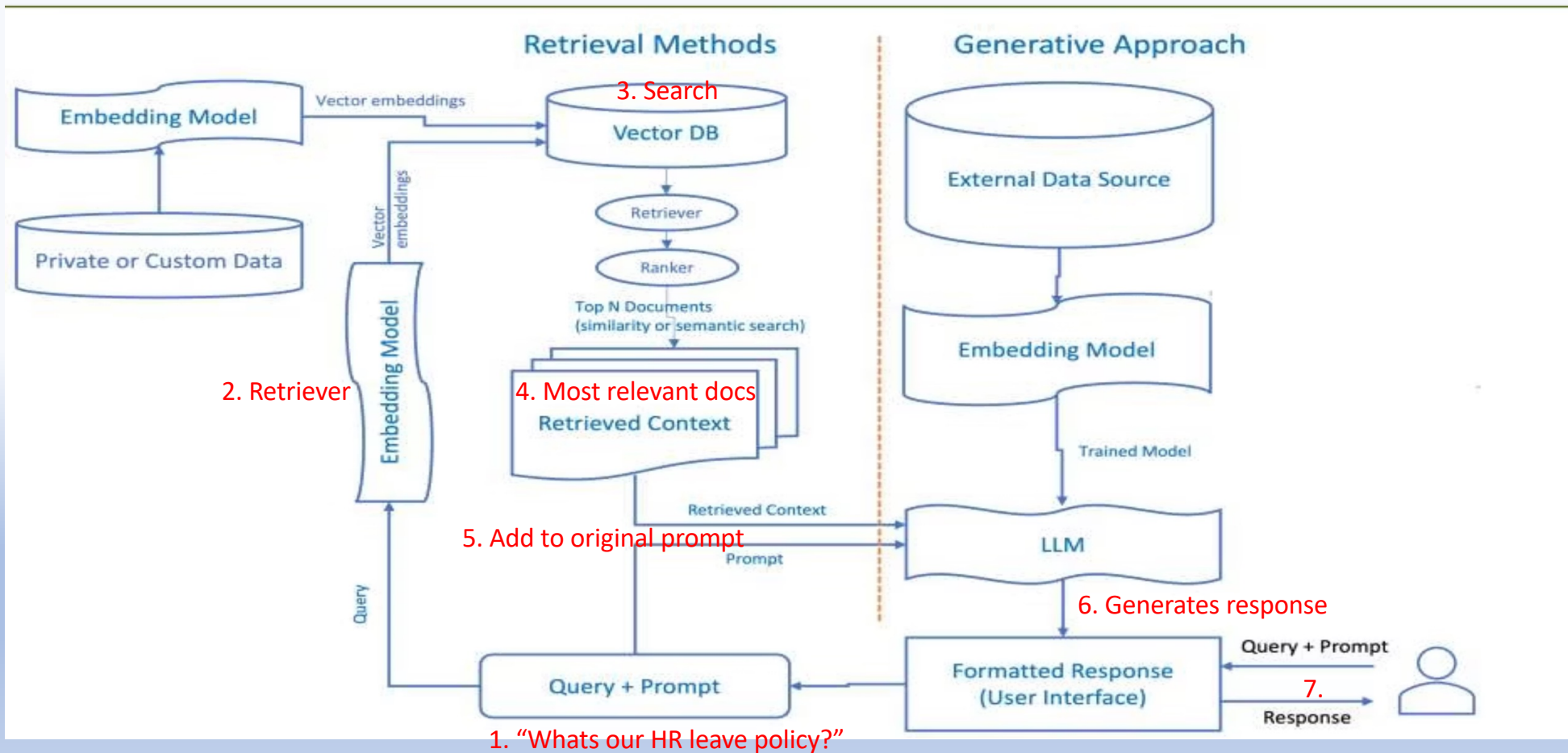


RAG Architecture - Generalized RAG Approach

Detective

Retrieval Augmented Generation

Storyteller



RAG Components

- RAG is an AI framework that allows a generative AI model to access external information not included in its training data or model parameters to enhance its responses to prompts.
- RAG seeks to combine the strengths of both retrieval-based and generative methods.
- It typically involves using a retriever component to fetch relevant passages or documents from a large corpus of knowledge.
- The retrieved information is then used to augment the generative model's understanding and improve the quality of generated responses.

RAG Components

- Retriever
- Ranker
- Generator
- External Data

What is a Prompt.

A prompt is the input provided by the user to generate a response. It could be a question, a statement, or any text that serves as the starting point for the model to generate a relevant and coherent continuation

RAG Components

Let's understand each component in detail

External data

The new data outside of the LLM's original training data set is called external data. It can come from multiple data sources, such as APIs, databases, or document repositories. The data may exist in various formats like files, database records, or long-form text.

Vector embeddings

- ML models cannot interpret information intelligibly in their raw format and require numerical data as input. They use neural network embeddings to convert real-world information into numerical representations called vectors.
- Vectors are numerical values that represent information in a multi-dimensional space
- Embedding vectors encode non-numerical data into a series of values that ML models can understand and relate. Example:

The Conference (Horror, 2023, Movie) *The Conference (1.2, 2023, 20.0)*

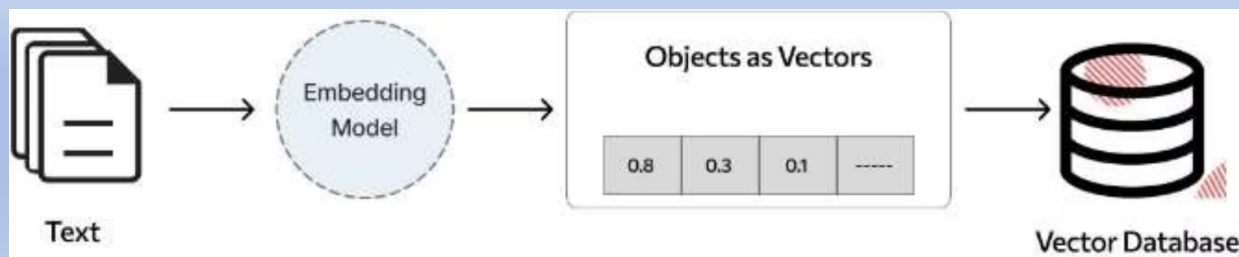
Tales from the Crypt (Horror, 1989, TV Show, Season 7) *Tales from the Crypt (1.2, 1989, 36.7)*

- The first number in the vector corresponds to a specific genre. An ML model would find that *The Conference* and *Tales from the Crypt* share the same genre. Likewise, the model will find more relationships

RAG Components

Vector DB

- Vector DB is a database that stores embeddings of words, phrases, or documents along with their corresponding identifiers.
- It allows for fast and scalable retrieval of similar items based on their vector representations.
- Vector DBs enable efficient retrieval of relevant information during the retrieval phase of RAG, improving the contextual relevance and quality of generated responses.
- **Data Chunking:** Before the retrieval model can search through the data, it's typically divided into manageable "chunks" or segments.
- Vector DB Examples : Chroma, Pinecone, Weaviate, Elasticsearch

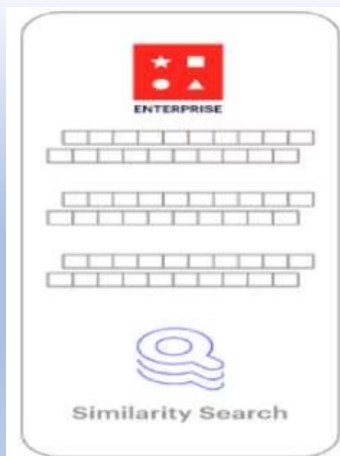


RAG Components

RAG Retriever

- The next step is to perform a relevancy search. The user query is converted to a vector representation and matched with the vector databases
- The retriever component is responsible for efficiently identifying and extracting relevant information from a vast amount of data.

For example, consider a smart chatbot for human resource questions for an organization. If an employee searches, *"How much annual leave do I have?"* the system will retrieve annual leave policy documents alongside the individual employee's past leave record. These specific documents will be returned because they are highly-relevant to what the employee has input. The relevancy was calculated and established using mathematical vector calculations and representations



RAG Components

RAG Ranker

- The RAG ranker component refines the retrieved information by assessing its relevance and importance. It assigns scores or ranks to the retrieved data points, helping prioritize the most relevant ones.
- The retriever component is responsible for efficiently identifying and extracting relevant information from a vast amount of data.

For example, consider a smart chatbot that can answer human resource questions for an organization. If an employee searches, *"How much annual leave do I have?"* the system will retrieve annual leave policy documents alongside the individual employee's past leave record.

Augment the LLM prompt

- Next, the RAG model augments the user input (or prompts) by adding the relevant retrieved data in context. This step uses prompt engineering techniques to communicate effectively with the LLM.
- The augmented prompt allows the large language models to generate an accurate answer to user queries.

RAG Components

RAG Generator

- The RAG generator component is basically the LLM Model such a (GPT)
- The RAG generator component is responsible for taking the retrieved and ranked original query, and generating the final response or output. information, along with the user's
- The generator ensures that the response aligns with the user's query and incorporates the factual knowledge retrieved from external sources.

Update external data

- To maintain current information for retrieval, asynchronously update the documents and update embedding representation of the documents.
- **Automated Real-time Processes:** Updates to documents and embeddings occur in real-time as soon as new information becomes available. This ensures that the system always reflects the most recent data.
- **Periodic Batch Processing:** Updates are performed at regular intervals (e.g., daily, weekly) in batches. This approach may be more efficient for systems with large volumes of data or where real-time updates are not necessary.

RAG Applications



Conversational AI

RAG enables chatbots to provide more accurate and contextually relevant responses to user queries.



Content Generation

In content generation tasks such as summarization, article writing, and content recommendation, RAG can augment the generation process with retrieved information, incorporating relevant facts, statistics, and examples from external sources.



Advanced Question & Answering

RAG enhances question answering systems by retrieving relevant passages or documents containing answers to user queries.



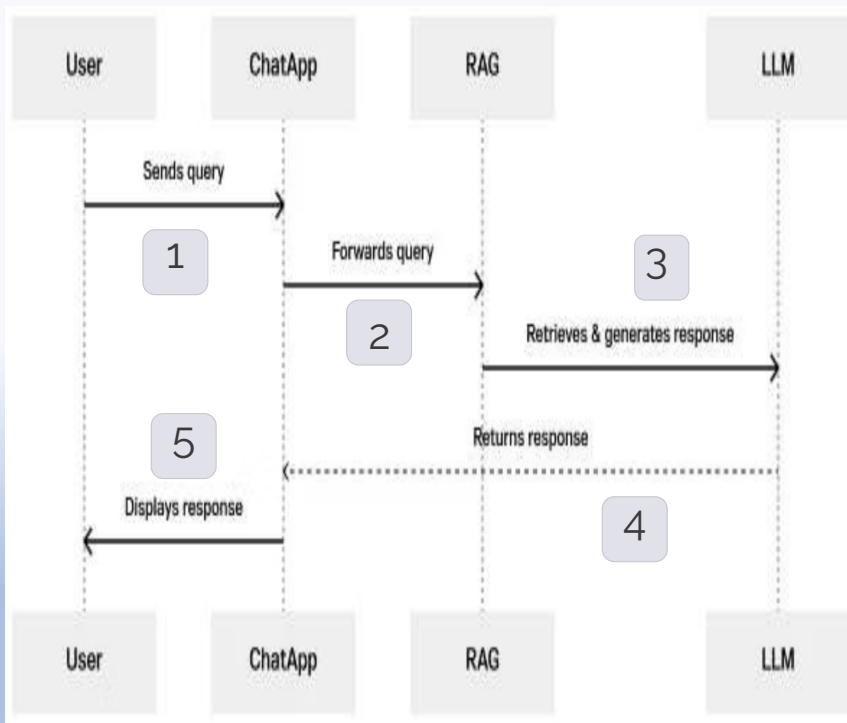
Healthcare

RAG can assist healthcare professionals in accessing relevant/latest medical literature, guidelines.

Legal!

Search tools that cite legal paragraphs

RAG Based Chat Application



Step 1 - User sends query

The process begins when the user sends a query or message to the chat application.

1

2

Step 2 - Chat App forwards query

Upon receiving the user's query, the chat application (Chat App) forwards this query to the Retrieval Augmented Generation (RAG) model for processing.

3

Step 3 - RAG retrieves + generates response

The RAG model, which integrates retrieval and generation capabilities, processes the user's query. It first retrieves relevant information from a large corpus of data, using the LLM to generate a coherent and contextually relevant response based on the retrieved information and the user's query.

4

Step 4 - LLM returns response

Once the response is generated, the LLM sends it back to the chat application (Chat App).

5

Step 5 - Chat App displays responses

Finally, the chat application displays the generated response to the user, completing the interaction.

RAG Benefits

1 Enhanced Relevance:

Incorporates external knowledge for more contextually relevant responses.

3 Versatility:

Adaptable to various tasks and domains without task-specific fine-tuning.

5 Dynamic Updates:

Allows for real-time or periodic updates to maintain current information.

7 Customization and Control:

Organizations can tailor the external sources RAG draws from, allowing control over the type and scope of information integrated into the model's responses.

2 Improved Quality:

Enhances the quality and accuracy of generated output.

4 Efficient Retrieval:

Leverages existing knowledge bases, reducing the need for large labeled datasets.

6 Trust and Transparency:

Accurate and reliable responses, underpinned by current and authoritative data, significantly enhance user trust in AI-driven applications.

8 Cost Effective

RAG Vs Fine Tuning

1 Objective

- Fine-tuning aims to adapt a pre-trained LLM to a specific task or domain by adjusting its parameters based on task-specific data.
- RAG focuses on improving the quality and relevance of generated text by incorporating retrieved information from external sources during the generation process.

3 Adaptability

- Fine-tuning makes the LLM more specialized and tailored to a specific task or domain
- RAG maintains the generalizability of the pre-trained LLM by leveraging external knowledge allowing it to adapt to a wide range of tasks

2 Training Data

- Fine-tuning requires task-specific labeled data/examples to update the model's parameters and optimize, leading to more time & cost
- RAG relies on a combination of pre-trained LLM and external knowledge bases

4 Model Architecture

- Fine-tuning typically involves modifying the parameters of the pre-trained LLM while keeping its architecture unchanged.
- RAG combines the retrieval and generation components, with the standard LLM architecture to incorporate the retrieval mechanism.

Useful Links

Youtube

- [How to Build a Local AI Agent With Python \(Ollama, LangChain & RAG\)](#)
- [Finally a Local RAG That WORKS!! \(+ FULL RAG Pipeline\)](#)
- [The BEST Way to Chunk Text for RAG](#)

Github

- <https://github.com/ThomasJanssen-tech/Local-RAG-with-Ollama>
- <https://github.com/ThomasJanssen-tech/Agentic-RAG-with-LangChain>
- <https://github.com/techwithtim/LocalAIAgentWithRAG>

Freedium

- [Building a Simple RAG System from Scratch: A Comprehensive Guide](#)
- [Chat with your documents tool – RAG \(vector DBs + cosine sim.\) & Claude API implementation](#)
- [RAG Without Embeddings? Here's how OpenAI is doing this...](#)

Datacamp

- [Retrieval Augmented Generation \(RAG\) with LangChain](#)

Files

main

Go to file

- .gitignore
- Building a Simple RAG System fro...
- Chat with your documents tool — ...
- RAG.py
- RAG_PDF.py
- RAG_PDF_Ollama.py**
- README.md

Marcvg69 RAG Init

Code Blame 38 lines (30 loc) · 1.16 KB

Raw Copy Download Edit History

```

1  from langchain_community.document_loaders import PyPDFLoader
2  from langchain_community.vectorstores import Chroma
3  from langchain_community.embeddings import OllamaEmbeddings
4  from langchain_community.llms import Ollama
5  from langchain.chains import RetrievalQA
6
7  # 1. Load PDF
8  pdf_path = "mydoc.pdf"
9  loader = PyPDFLoader(pdf_path)
10 docs = loader.load_and_split()
11
12 # 2. Create vectorstore (embedding the documents)
13 embedding = OllamaEmbeddings(model="nomic-embed-text") # You must have this model pulled
14 vectordb = Chroma.from_documents(docs, embedding=embedding, persist_directory="./chroma_db")
15
16 # 3. Define the retriever
17 retriever = vectordb.as_retriever()
18
19 # 4. Load local LLM from Ollama
20 llm = Ollama(model="llama3") # Change to "mistral" or another if you prefer
21
22 # 5. Build RetrievalQA pipeline
23 qa_chain = RetrievalQA.from_chain_type(
24     llm=llm,
25     retriever=retriever,
26     return_source_documents=True
27 )
28
29 # 6. Run a chat loop
30 print("Ask me anything about the PDF. Type 'exit' to quit.")
31 while True:
32     question = input("\nYou: ")
33     if question.lower() in ["exit", "quit"]:
34         break
35
36     result = qa_chain({"query": question})
37     print(f"\n▀ Answer:\n{result['result']}")

```